TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

Semestre: Enero – Agosto 2026

CARRERA
Ingeniería Sistemas Computacionales

MATERIA
Patrones de diseño de software

TÍTULO
Composite

UNIDAD
3

ALUMNO(S) Y NO. DE CONTROL
Esmeralda Gamboa Hernandez
21210374
Velazquez Morales Lucero
-22210362
Vidales Hurtado Alitzel Dalhi
-21212371
Valdes Siqueiros Ivan Antonio
21210430

DOCENTE
MARIBEL GUERRERO LUIS

Tijuana Baja,California a 26 de Marzo del 2026

Introducción

El patrón Bridge se centra en desacoplar una abstracción de su implementación para que ambas puedan evolucionar de forma independiente, utilizando la composición en lugar de la herencia para evitar una explosión de clases. Por otro lado, el patrón Decorator permite añadir funcionalidades o responsabilidades adicionales a un objeto de manera dinámica y transparente, envolviéndolo en una serie de clases decoradoras sin alterar su estructura original. Mientras que el Bridge se utiliza principalmente en la fase de diseño para separar interfaces de plataformas o motores de ejecución, el Decorador es ideal para extender el comportamiento de objetos individuales en tiempo de ejecución sin afectar a otros objetos de la misma clase.

Código

```
using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;


namespace ProyectoEquipo

{



  //Clases asbstracta princial

  public abstract class TecnmComponent

  {

    public abstract string Clave { get; }

    public abstract string Descripcion { get; }

  }
```

```
//Decorator abstracto
public abstract class AgregadoDecorator : TecnmComponent
{
    protected TecnmComponent _tecnm;

    public AgregadoDecorator(TecnmComponent tecnm)
    {
        _tecnm = tecnm;
    }
}

//Definicion de carreras

public class IngI : TecnmComponent
{
    public override string Clave => "INF";
    public override string Descripcion => "Ingenieria en Informatica";
}

public class IngC : TecnmComponent
{
    public override string Clave => "IC";
    public override string Descripcion => "Ingenieria en Ciberseguridad";
}

public class IngSC : TecnmComponent
{
    public override string Clave => "ISC";
    public override string Descripcion => "Ingenieria en Sistemas Computacionales";
```

```
}

// Materiales (Bridge)

public interface IMaterial//Este siendo el puente entre el decorador y los materiales, ya que el decorador no conoce los detalles de los materiales, solo sabe que tiene una descripción

{

    string ObtenerDescripcion();

}


public class Libro : IMaterial

{

    private string nombre;


    public Libro(string nombre)

    {

        this.nombre = nombre;

    }


    public string ObtenerDescripcion()

    {

        return "Libro: " + nombre;

    }

}


//Clases de materias (Decorador)

public class Calculo : AgregadoDecorator

{

    private List<IMaterial> materiales = new List<IMaterial>();


    public Calculo(TecnmComponent tecnm) : base(tecnm)
```

```
    {
        materiales.Add(new Libro("Calculo Stewart"));
        materiales.Add(new Libro("Calculo Larson"));
    }

    public override string Clave => _tecnm.Clave + "C1S";

    public override string Descripcion
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            sb.AppendLine(_tecnm.Descripcion + ", Materia: Calculo");

            foreach (var m in materiales)
                sb.AppendLine("   - " + m.ObtenerDescripcion());

            return sb.ToString();
        }
    }
}

public class Fisica : AgregadoDecorator
{
    private List<IMaterial> materiales = new List<IMaterial>();

    public Fisica(TecnmComponent tecnm) : base(tecnm)
    {
        materiales.Add(new Libro("Fisica Vol 1"));
```

```
        materiales.Add(new Libro("Fisica Vol 2"));
    }

    public override string Clave => _tecnm.Clave + "F1S";

    public override string Descripcion
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            sb.AppendLine(_tecnm.Descripcion + ", Materia: Fisica");

            foreach (var m in materiales)
                sb.AppendLine("   - " + m.ObtenerDescripcion());

            return sb.ToString();
        }
    }
}

public class Programacion : AgregadoDecorator
{
    private List<IMaterial> materiales = new List<IMaterial>();

    public Programacion(TecnmComponent tecnm) : base(tecnm)
    {
        // Aquí definen los materiales (Bridge)
        materiales.Add(new Libro("C# Basico"));
        materiales.Add(new Libro("POO en C#"));
```

```
    }

    public override string Clave => _tecnm.Clave + "P1S";

    public override string Descripcion
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            sb.AppendLine(_tecnm.Descripcion + ", Materia: Programacion");

            foreach (var m in materiales)
                sb.AppendLine("   - " + m.ObtenerDescripcion());

            return sb.ToString();
        }
    }
}


internal class Program
{
    static void Main(string[] args)
    {

        TecnmComponent carrera = null;

        // MENÚ DE CARRERA
        Console.WriteLine("Seleccione una carrera:");
```

```
Console.WriteLine("1. Ingenieria en Sistemas Computacionales");
Console.WriteLine("2. Ingenieria en Informatica");
Console.WriteLine("3. Ingenieria en Ciberseguridad");

int opcionCarrera = int.Parse(Console.ReadLine());

switch (opcionCarrera)
{
    case 1:
        carrera = new IngSC();
        break;
    case 2:
        carrera = new IngI();
        break;
    case 3:
        carrera = new IngC();
        break;
    default:
        Console.WriteLine("Opcion invalida");
        return;
}

int opcion;

// MENÚ DE MATERIAS
do
{
    Console.WriteLine("\nSeleccione una materia:");
    Console.WriteLine("1. Fisica");
```

```
            Console.WriteLine("2. Calculo");
            Console.WriteLine("3. Programacion");
            Console.WriteLine("0. Terminar");

            opcion = int.Parse(Console.ReadLine());

            switch (opcion)
            {
                case 1:
                    carrera = new Fisica(carrera);
                    break;
                case 2:
                    carrera = new Calculo(carrera);
                    break;
                case 3:
                    carrera = new Programacion(carrera);
                    break;
            }

        } while (opcion != 0);

        Console.WriteLine("\n===== RESULTADO FINAL =====\n");
        Console.WriteLine(carrera.Descripcion);
        Console.WriteLine("Clave: " + carrera.Clave);

        Console.ReadKey();

    }
}
```

}

CAPTURAS

```
Seleccione una carrera:
1. Ingenieria en Sistemas Computacionales
2. Ingenieria en Informatica
3. Ingenieria en Ciberseguridad
```

Al iniciar el programa se podrá ver la siguiente pantalla donde el usuario tendrá que elegir una de las 3 opciones

```
Seleccione una materia:
1. Fisica
2. Calculo
3. Programacion
0. Terminar
```

Ya que el usuario haya elegido su carrera podrá elegir cuantas materias desea tener puede tomar cuantas quiera, si decide una de las 3 opciones se le volverá a mostrar el mismo menú hasta que el usuario seleccione la opción de terminar.

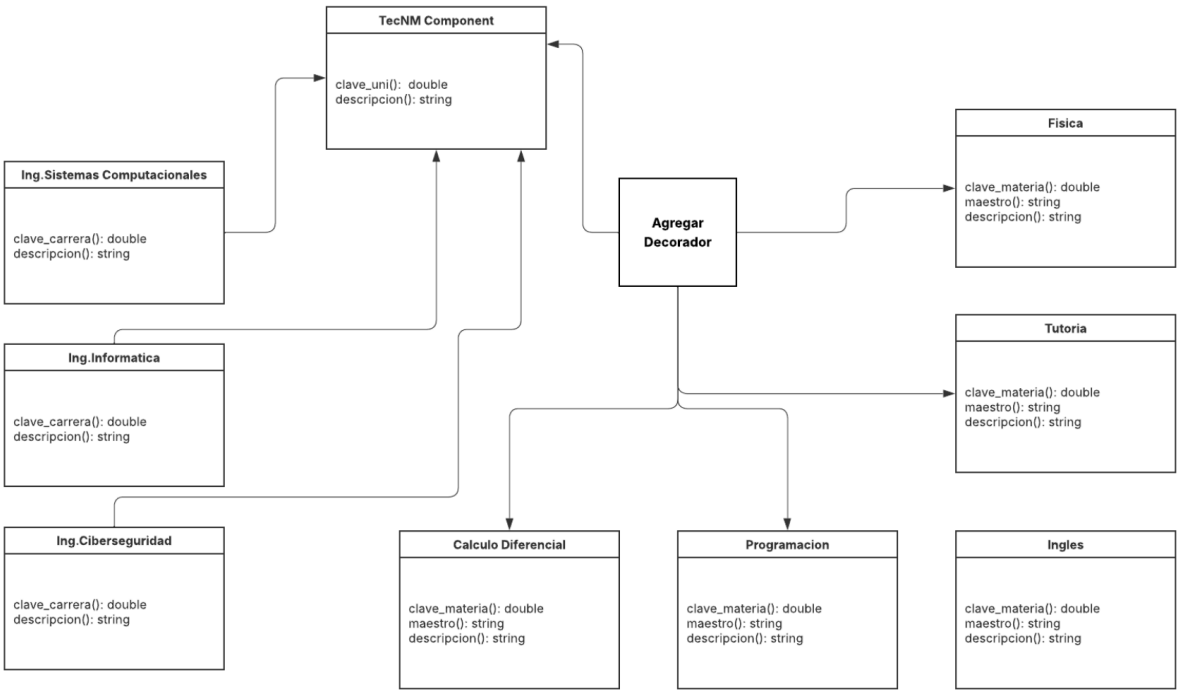
```
===== RESULTADO FINAL =====

Ingenieria en Informatica, Materia: Fisica
   - Libro: Fisica Vol 1
   - Libro: Fisica Vol 2

Clave: INFF1S
```

Una vez que el usuario haya elegido la opción de terminar se despliega un menú donde se puede observar la carrera y las diferentes materias que eligió tomar con su clave y sus respectivos libros, para salir del programa solamente se requiere darle a la tecla enter.

UML



Conclusiones

Esmeralda Gamboa Hernandez

-Esta práctico me ayudó a comprender los patrones Decorator y Bridge, en conclusión el decorator funciona para que en el sistema se le agreguen clases pero sin cambiar las clases originales, sería el agregarle las materias, esos serían nuestros decoradores. Por otro lado , el patrón Bridge funciona para que los datos o información de una clase se imprementen a nuestro sistema de una manera que no dañe lo anterior realizado, en nuestro caso lo que hicimos fue agregarle a las materias paquetes de libros para que a la hora que el usuario imprima su información le aparezca su carrera, materias y paquetes de libros de estas.

Velazquez Morales Lucero

-En este proyecto pude comprender mejor cómo funcionan los patrones de diseño Decorator y Bridge al aplicarlos al proyecto. El patrón Decorator me permitió agregar materias a una carrera de forma dinámica, sin necesidad de modificar las clases base, lo que hace que el sistema sea más flexible y fácil de entender. Por otro lado, el patron Bridge me ayudó a separar las materias de sus recursos, en este caso los libros, permitiendo que cada materia tenga sus propios materiales sin depender directamente de ellos. Al combinar ambos patrones me ayudó entender la importancia de diseñar sistemas desacoplados y bien estructurados.

Vidales Hurtado Alitzel Dalhi

-Ambos patrones tienen una estructura bastante similar sin embargo sirven para aspectos diferentes, en la elaboración de este proyecto Decorator ayudó con la base de las materias haciendo más fácil que a futuro se puedan agregar tanto más

materias como características de estas mismas y Bridge nos ayudó a asignarle los libros correspondientes a cada una de las materia. Al inicio fue un poco complicado pensar en cómo aplicariamls correctamente ambos patrones pero consideró que lo logramos de una manera eficiente para el sistema.

Siqueiros Valdes Ivan Antonio
-El Patrón bridge lo veo muy practico y sencillo en su aplicacion asi como su utilidad, ya que simplemente separa el cómo se hace algo del que es para que asi el como se pueda hacer de múltiples maneras, mientras el decorador en la aplicación es más complejo siendo que en su definición suena muy sencillo, el último permitiendo que se puedan extender las clases en vez de modificarlas.